

BÁO CÁO THỬ NGHIỆM: KỸ NĂNG VIẾT PROMPT TRONG HỌC TẬP CHUYÊN NGÀNH

Họ và tên: Trần Văn Tài Anh

Lớp: [VNU1001_E252012] - Trường Đại học Công nghệ (UET)

Chủ đề thử nghiệm: Thuật toán Quy hoạch động (Dynamic Programming)

I. Lựa chọn tác vụ học tập

Trong báo cáo này, tôi chọn tác vụ: **Giải thích một khái niệm phức tạp.**

- Khái niệm:** Thuật toán Quy hoạch động (Dynamic Programming - DP).
- Lý do lựa chọn:** Đây là một mảng kiến thức khó trong môn Cấu trúc dữ liệu và Giải thuật. Việc hiểu sai bản chất sẽ dẫn đến khó khăn trong việc cài đặt code. Tôi muốn thử nghiệm xem các mức độ prompt khác nhau sẽ giúp một sinh viên CNTT tiếp cận khái niệm này hiệu quả ra sao.

II. Xây dựng các phiên bản Prompt

Tôi đã xây dựng 3 phiên bản prompt với độ chi tiết và kỹ thuật tăng dần:

- Prompt Cơ bản (Đơn giản, ngắn gọn):** "Giải thích cho tôi thế nào là quy hoạch động trong lập trình."
- Prompt Cải tiến (Chi tiết hơn, có cấu trúc):** "Hãy đóng vai một trợ giảng môn Cấu trúc dữ liệu và Giải thuật. Giải thích khái niệm Quy hoạch động (Dynamic Programming), nêu sự khác biệt giữa nó và thuật toán Đệ quy thông thường. Cho một ví dụ minh họa đơn giản."
- Prompt Nâng cao (Áp dụng kỹ thuật Role-play, Chain-of-Thought, Constraints):** "Bạn là một chuyên gia thuật toán. Hãy giải thích bản chất của Quy hoạch động (DP) cho một sinh viên đang học giải thuật. Yêu cầu: 1. Phân tích 2 đặc điểm cốt lõi... 2. So sánh Top-down và Bottom-up... 3. Viết mã giả cho bài toán LIS... 4. Giải thích tại sao DP tối ưu hơn đệ quy thuần..."

III. Thử nghiệm và So sánh kết quả

1. Kết quả thử nghiệm

Dưới đây là minh chứng hình ảnh từ quá trình thử nghiệm với AI:

- Thử nghiệm Prompt 1:**

Giải thích cho tôi thế nào là quy hoạch động trong lập trình.

Quy hoạch động (Dynamic Programming – DP) là một kỹ thuật trong lập trình dùng để giải các bài toán có **tính chất lặp lại** và có thể chia nhỏ thành các bài toán con.

Hiểu đơn giản:

👉 Nếu một bài toán lớn có thể tách thành nhiều bài nhỏ giống nhau, và các bài nhỏ đó bị tính đi tính lại nhiều lần → ta lưu lại kết quả để dùng lại → đó là quy hoạch động.

1. Ý tưởng cốt lõi

DP dựa trên 2 đặc điểm chính:

♦ 1. Tính chất chồng lặp (Overlapping Subproblems)

Các bài toán con bị lặp lại nhiều lần.

Ví dụ:

- Tính Fibonacci:

```
fib(5) = fib(4) + fib(3)
fib(4) = fib(3) + fib(2)
```



👉 fib(3) bị tính lại nhiều lần → rất tốn thời gian

♦ 2. Tính chất cấu trúc tối ưu (Optimal Substructure)

Lời giải của bài toán lớn có thể xây dựng từ lời giải tối ưu của các bài toán con.

2. Cách hoạt động

Thay vì tính đi tính lại → ta lưu kết quả lại.

Có 2 cách chính:

♦ Cách 1: Memoization (Top-down)

- Dùng đệ quy
- Lưu kết quả vào mảng / map

Ví dụ Fibonacci:

```
<> Java
int[] dp = new int[100];

int fib(int n) {
    if (n <= 1) return n;
    if (dp[n] != 0) return dp[n]; // đã tính rồi

    dp[n] = fib(n-1) + fib(n-2);
    return dp[n];
}
```



♦ Cách 2: Tabulation (Bottom-up)

- Không dùng đệ quy
- Tính từ nhỏ → lớn

```
<> Java
int fib(int n) {
    int[] dp = new int[n+1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }

    return dp[n];
}
```



3. Khi nào dùng quy hoạch động?

Dùng khi:

- Bài toán có **lặp lại kết quả**
- Có thể chia thành các bài toán con
- Có công thức truy hồi (recurrence)

4. Ví dụ dễ hiểu

Bài toán cái ba lô (Knapsack)

- Có nhiều đồ vật (trọng lượng, giá trị)
- Ba lô có giới hạn cân nặng
- Chọn đồ sao cho **giá trị lớn nhất**

👉 Đây là bài kinh điển của DP

5. So sánh nhanh

Cách	Ý tưởng	Tốc độ
Đệ quy thường	Tính lại nhiều lần	Rất chậm
Quy hoạch động	Lưu lại kết quả	Nhanh hơn rất nhiều

6. Tóm lại

👉 Quy hoạch động =

Chia nhỏ bài toán + Lưu kết quả + Dùng lại

Nếu bạn muốn, tôi có thể:

- Cho bạn **lộ trình học DP từ dễ → khó (rất quan trọng)**
- Hoặc đưa bài tập đúng level của bạn (vì bạn đã  học OOP Java rồi, có thể bắt đầu DP luôn)

     ...

• Thử nghiệm Prompt 2:

Hãy đóng vai một trợ giảng môn Cấu trúc dữ liệu và Giải thuật. Giải thích khái niệm Quy hoạch động (Dynamic Programming), nêu sự khác biệt giữa nó và thuật toán Đệ quy thông thường. Cho một ví dụ minh họa đơn giản.

Được rồi, mình sẽ giải thích theo kiểu trợ giảng — rõ bản chất, dễ hiểu, không học vẹt.

1. Quy hoạch động (Dynamic Programming) là gì?

Quy hoạch động (DP) là một kỹ thuật giải bài toán bằng cách:

- 👉 Chia bài toán lớn thành các bài toán con nhỏ hơn
- 👉 Lưu lại kết quả các bài toán con để không tính lại

Nói ngắn gọn:

DP = Chia nhỏ + Ghi nhớ (memoization) + Tái sử dụng

2. Khi nào dùng Quy hoạch động?

Một bài toán phù hợp với DP khi có 2 đặc điểm:

- ✅ 1. Bài toán con chồng lặp (Overlapping Subproblems)
 - Cùng một bài toán con bị gọi lại nhiều lần
- ✅ 2. Cấu trúc con tối ưu (Optimal Substructure)
 - Lời giải tối ưu của bài toán lớn được xây từ lời giải tối ưu của bài toán con

3. So sánh với Đệ quy thông thường

Tiêu chí	Đệ quy thường	Quy hoạch động
Cách làm	Gọi lại hàm liên tục	Gọi + lưu kết quả
Tính lại	❌ Rất nhiều	✅ Không
Tốc độ	Chậm (có thể exponential)	Nhanh hơn nhiều
Bộ nhớ	Ít	Tốn thêm để lưu

👉 Điểm khác biệt cốt lõi:

Đệ quy: tính lại
DP: nhớ lại

4. Ví dụ kinh điển: Fibonacci

❌ Cách đệ quy thường

```
<> Java
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2);
}
```

Vấn đề:

- fib(5) sẽ gọi lại fib(3) rất nhiều lần
- Độ phức tạp: $O(2^n)$ (rất chậm)

✅ Cách dùng Quy hoạch động

Cách 1: Memoization (Top-down)

```
Java

int[] dp = new int[1000];

int fib(int n) {
    if (n <= 1) return n;
    if (dp[n] != 0) return dp[n];

    dp[n] = fib(n-1) + fib(n-2);
    return dp[n];
}
```

Cách 2: Tabulation (Bottom-up)

```
Java

int fib(int n) {
    int[] dp = new int[n+1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }

    return dp[n];
}
```

5. Tư duy đúng khi học DP

Khi gặp bài toán, hãy tự hỏi:

1. Bài toán con là gì?
2. Công thức truy hồi (recurrence) là gì?
3. Có bị tính lại không?
4. Lưu kết quả ở đâu? (mảng dp)

6. Tóm lại (cốt lõi để nhớ)

- 🔥 Quy hoạch động không phải là thuật toán cụ thể
- 👉 Nó là cách tư duy tối ưu hóa đệ quy bằng cách tránh tính lại

Nếu bạn muốn, mình có thể:

- Cho bài tập từ dễ → khó (rất quan trọng để hiểu thật)
- Hoặc giải thích cách **nhìn bài ra DP trong 5 giây** (cái này là kỹ năng thi rất mạnh)

Bạn muốn đi theo hướng nào?

- Thử nghiệm Prompt 3:

Bạn là một chuyên gia thuật toán. Hãy giải thích bản chất của Quy hoạch động (DP) cho một sinh viên đang học giải thuật.

Yêu cầu:

- Phân tích 2 đặc điểm cốt lõi: Optimal Substructure (Cấu trúc con tối ưu) và Overlapping Subproblems (Bài toán con gối nhau).
- So sánh cách tiếp cận Top-down (Memoization) và Bottom-up (Tabulation).
- Viết mã giả (Pseudocode) cho bài toán dãy con tăng dài nhất (LIS) bằng DP.
- Giải thích tại sao DP lại tối ưu hơn Đệ quy thuần túy về độ phức tạp thời gian.

Mình sẽ giải thích đúng "bản chất" theo góc nhìn của người làm thuật toán, không chỉ dừng ở định nghĩa.

1. Hai đặc điểm cốt lõi của DP

◆ 1. Optimal Substructure (Cấu trúc con tối ưu)

Một bài toán có cấu trúc con tối ưu khi:

Lời giải tối ưu của bài toán lớn có thể được xây dựng từ lời giải tối ưu của các bài toán con.

Ví dụ trực quan (LIS):

Ví dụ trực quan (LIS):

Giả sử bạn đang tìm dãy con tăng dài nhất kết thúc tại vị trí i .

- Nếu phần tử trước đó là $j < i$ và $a[j] < a[i]$
- Thì:

$$\text{LIS tại } i = \text{LIS tại } j + 1$$

➡ Nghĩa là:

- Bạn không cần giải lại từ đầu
- Chỉ cần dùng kết quả tối ưu đã có trước đó

♦ 2. Overlapping Subproblems (Bài toán con chồng lặp)

Một bài toán có tính chất này khi:

Cùng một bài toán con bị giải lại nhiều lần.

Ví dụ:

Trong Fibonacci:

```
fib(5)
= fib(4) + fib(3)
= (fib(3) + fib(2)) + fib(3)
```



👉 `fib(3)` bị tính lại nhiều lần

🔥 Kết luận bản chất:

DP hoạt động hiệu quả vì:

- “Bài toán có thể chia nhỏ hợp lệ (optimal substructure)”
- “Và có lặp lại để tối ưu (overlapping subproblems)”

2. Top-down vs Bottom-up

◆ Top-down (Memoization)

👉 Cách làm:

- Viết đệ quy như bình thường
- Thêm cache (mảng/map) để lưu kết quả

Đặc điểm:

- Tự nhiên, dễ viết
- Chỉ tính những trạng thái cần thiết

Pseudocode:

```
function solve(x):  
    if x đã có trong dp:  
        return dp[x]  
  
    dp[x] = tính từ các bài toán con  
    return dp[x]
```



◆ Bottom-up (Tabulation)

👉 Cách làm:

- Bỏ đệ quy
- Tính từ nhỏ → lớn

Đặc điểm:

- Không dùng stack → tránh tràn stack
- Kiểm soát tốt thứ tự tính

Pseudocode:

```
dp[base cases]  
  
for trạng thái từ nhỏ → lớn:  
    dp[i] = tính từ trạng thái trước đó
```



 So sánh nhanh

Tiêu chí	Top-down	Bottom-up
Dễ viết	✓	✗ (cần nghĩ thứ tự)
Stack	Dùng đệ quy	Không
Tối ưu	Có thể dư trạng thái	Tối ưu toàn bộ
Debug	Dễ hơn	Khó hơn

3. Pseudocode: Longest Increasing Subsequence (LIS)

Ý tưởng:

- `dp[i]` = độ dài dãy con tăng dài nhất kết thúc tại `i`
- Với mỗi `i`, xét tất cả `j < i`

Mã giả ($O(n^2)$):

```
input: array a[0..n-1]

dp[i] = 1 với mọi i

for i từ 0 → n-1:
    for j từ 0 → i-1:
        if a[j] < a[i]:
            dp[i] = max(dp[i], dp[j] + 1)

kết quả = max(dp[i] với mọi i)
```

Giải thích:

- Mỗi phần tử tự nó là 1 dãy → `dp[i] = 1`
- Nếu nối được từ `j → i` thì cập nhật

4. Vì sao DP nhanh hơn đệ quy thuần?

❌ Đệ quy thuần

- Gọi lại cùng một bài toán nhiều lần
- Số lần gọi có thể tăng **theo cấp số nhân**

👉 Ví dụ Fibonacci:

- Độ phức tạp: $O(2^n)$

✅ DP

- Mỗi trạng thái chỉ tính **1 lần**
- Sau đó chỉ **tra cứu lại** ($O(1)$)

👉 Tổng số trạng thái:

- Fibonacci: n trạng thái $\rightarrow O(n)$
- LIS: n^2 trạng thái $\rightarrow O(n^2)$

🔥 Insight quan trọng:

Đệ quy thuần:

- “Thời gian = số lần gọi hàm (rất lớn)”

DP:

- “Thời gian = số trạng thái $\times$ thời gian chuyển trạng thái”

5. Tư duy “chuẩn DP” (cái này quan trọng nhất)

Khi gặp bài:

1. State là gì? ($dp[i]$, $dp[i][j]$, ...)
2. Transition (công thức truy hồi)
3. Base case
4. Thứ tự tính

6. Tóm lại (đúng bản chất)

DP không phải là “mẹo”



👉 Nó là cách biến một lời giải đệ quy exponential thành polynomial bằng cách lưu trạng thái

2. Bảng so sánh kết quả

Tiêu chí so sánh	Prompt 1 (Cơ bản)	Prompt 2 (Cải tiến)	Prompt 3 (Nâng cao)
Độ sâu kiến thức	Chỉ dừng ở định nghĩa và ví dụ Fibonacci cơ bản.	Bắt đầu có sự so sánh về tốc độ và bộ nhớ.	Phân tích sâu về bản chất "biến lờỉ giải exponential thành polynomial".
Cấu trúc trình bày	Liệt kê các mục đơn giản.	Có bảng so sánh, icon nhận diện, dễ theo dõi.	Cấu trúc chuyên nghiệp, có mã giả (pseudocode) bài toán khó (LIS).
Kỹ thuật Prompt	Không sử dụng kỹ thuật nào.	Sử dụng Role Prompting (Trợ giảng).	Kết hợp Role Play , Few-shot (gợi ý bài toán), Constraints .
Tính ứng dụng	Chỉ để đọc cho biết.	Giúp sinh viên hiểu cách tư duy khi gặp bài toán mới.	Cung cấp công cụ mạnh để ôn thi và cài đặt thuật toán thực tế.

IV. Phân tích lý do hiệu quả của các Prompt

Dựa trên kết quả thực tế, tôi nhận thấy sự vượt trội của Prompt Nâng cao nhờ các yếu tố sau:

- Thiết lập vai trò (Role-play):** Khi đóng vai "Chuyên gia thuật toán", AI chuyển từ giọng văn giải thích từ điển sang giọng văn phân tích kỹ thuật, tập trung vào các thông số như độ phức tạp $O(n^2)$ thay vì chỉ nói "nhanh hơn".
- Ràng buộc cụ thể (Constraints):** Việc yêu cầu mã giả cho bài toán LIS thay vì Fibonacci (quá phổ biến) buộc AI phải truy xuất dữ liệu chuyên sâu hơn, giúp người học tiếp cận được các bài toán thực tế trong kỳ thi tại UET.
- Kỹ thuật Chain-of-Thought:** Bằng cách yêu cầu phân tích từng bước (từ đặc điểm cốt lõi đến so sánh cách tiếp cận), AI đưa ra một lộ trình tư duy logic giúp người học dễ dàng hình dung quy trình giải quyết bài toán DP: **State -> Transition -> Base case -> Order**.
- Sự tương tác chuyên sâu:** Ở Prompt 3, AI còn đưa ra được "Insight quan trọng" và "Tư duy chuẩn DP", điều mà các prompt ngắn không bao giờ chạm tới được.

V. Tổng hợp nguyên tắc và mẹo viết Prompt hiệu quả

Qua quá trình thực nghiệm, tôi đúc kết được bộ nguyên tắc viết prompt để tối ưu hóa việc học tập:

- Nguyên tắc 1: Cung cấp bối cảnh (Context is King).** Luôn cho AI biết nó là ai và người nhận tin là ai (Ví dụ: "Tôi là sinh viên UET đang ôn thi giải thuật").
- Nguyên tắc 2: Ràng buộc định dạng (Formatting).** Yêu cầu AI trình bày dưới dạng bảng, mã giả hoặc danh sách để dễ dàng tra cứu.
- Nguyên tắc 3: Sử dụng các bài toán mục tiêu.** Thay vì hỏi chung chung, hãy yêu cầu AI áp dụng khái niệm vào một bài toán cụ thể để kiểm tra độ hiểu (Ví dụ: bài toán LIS, Cái túi...).

- **Nguyên tắc 4: Tinh chỉnh dần (Iterative).** Nếu AI trả lời chưa sâu, hãy đặt câu hỏi nối tiếp dựa trên những từ khóa chuyên môn AI vừa cung cấp (như "Tabulation" hay "Memoization").

Kết luận: Viết prompt không chỉ là đưa ra câu lệnh, mà là quá trình tư duy để điều khiển AI phục vụ mục đích học tập. Một prompt tốt có thể thay thế hàng giờ tìm kiếm tài liệu rời rạc trên mạng.